

Week-1 module time complexity analysis

1) Square pattern

↳ Time complexity: $O(n^2)$

The inner loop runs n times for each of the n rows.
total operations = $n \times n = n^2$

↳ Space complexity: $O(1)$

↳ we are not storing any value.

2) Hollow Square, Right triangle, inverted triangle, pyramid, Diamond, Half-Diamond, Half-pyramid inverted, Number-Square, incrementing number pattern, RTN, inverted diamond number, sandwich pattern, incrementing diamond pattern.

↳ All have same explanation like above.

↳ nested loops have: $O(n^2)$ time complexity.

Week 2 Module

1) Linear Search

Complexity

- Best: $O(1)$
- Avg: $O(n)$
- Worst: $O(n)$
- Space: $O(1)$

Teacher's Signature.....

* Explanation.

→ check elements one by one until target is found.

* Optional Solution.

- If array is unsorted, linear-search is already optimal
- If many searches are performed, use Hash-Table $\rightarrow O(1)$ avg lookup

2) Binary Search

Complexity

- Best: $O(1)$
- Avg: $O(\log n)$
- Worst: $O(\log n)$
- Space:
iterative: $O(1)$, Recursive: $O(\log n)$

* Explanation

→ Search space becomes half every iteration.

* Optional Solution

→ Binary Search is already optimal for sorted arrays

3) Bubble-Sort

Complexity

without optimisation

- Best: $O(n^2)$
- Avg: $O(n^2)$
- Worst: $O(n^2)$

Teacher's Signature.....

optimal sol.
few real application use.

- Merge Sort $\rightarrow O(n \log n)$
- Quick Sort $\rightarrow$ Avg $O(n \log n)$
- Heap Sort $\rightarrow O(n \log n)$

4) Insertion sort:-

Complexity

- Best: $O(n)$
- Avg: $O(n^2)$
- Worst: $O(n^2)$

Space: $O(1)$

Explanation

Excellent for nearly sorted arrays

Optimal Solution

- Nearly sorted $\rightarrow$ insertion sort
- Large random data $\rightarrow$ merge / quick sort.

5) Selection Sort

complexity

- Best: $O(n^2)$
- Avg: $O(n^2)$
- Worst: $O(n^2)$
- Space: $O(1)$

Explanation

Teacher's Signature.....

find minimum every iteration.

* optimal sol

Not efficient for large dataset

i) Merge Sort

complexity

Explanation

Best: $O(n \log n)$
 Avg: $O(n \log n)$
 Worst: $O(n \log n)$
 Space: $O(n)$

Explanation:-

Divide array into halves
 and merge sorted halves

Optimal Sol

merge sort itself is optimal

ii) Merge Sort (max)

operation after
 sorting

Median $\rightarrow O(1)$

(count > median $\rightarrow O(n)$)

diff. = max - min
 $O(1)$

Overall

sorting complexity

$O(n \log n)$

iii) Quick Sort (Easy)

Best: $O(n \log n)$

Avg: $O(n \log n)$

Worst: $O(n^2)$

Space: $O(\log n)$ avg.

Explanation:-

Partition around a pivot

Optimal solution

Use randomised or median-of-three
 pivot to reduce the chance of
 the worst case.

iv) Quick Sort (med)

$O(n \log n)$

Teacher's Signature.....

Heap sort

Best: $O(n \log n)$

Avg: $O(n \log n)$

Worst: $O(n \log n)$

Space: $O(1)$

Explanation:

Build Heap, then repeatedly
remove the maximum
element

Optional Ed:

Heap sort is already
optional when constant extra
space is req.

Heap Sort (Med)

Fastest - $O(1)$

Slowest - $O(1)$

Avg - $O(n)$

Count below avg - $O(n)$

percentage - $O(1)$

Overall

$O(n \log n)$

Teacher's Signature.....


```

* #include <iostream>
using namespace std;
int main() {
    int N;
    cin >> N;
    for (int i = 1; i <= N; i++) {
        cout << i << " ";
    }
    return 0;
}

```

- i) Time complexity
 $O(N)$
- ii) Space complexity
 $O(1)$

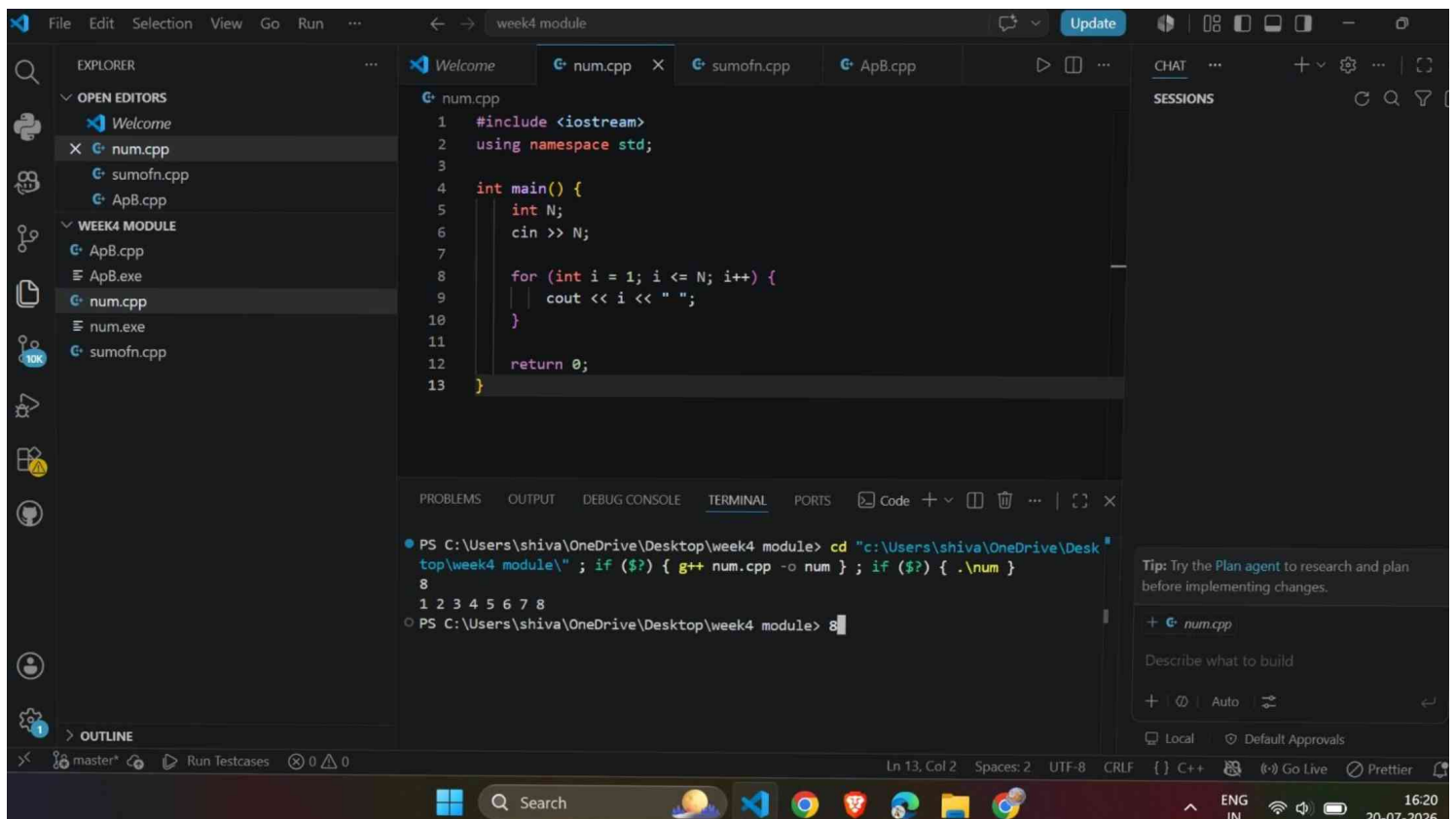
Explanation:

- ↳ The loop runs from 1 to N exactly once.
- ↳ Each iteration prints one number, so the total work is proportional to N.
- ↳ No extra data structures are used, so only constants extra memory is required.

Optimised approach

- ↳ This is already the optimal solution because every number from 1 to N must be printed.

Teacher's Signature.....



2) Sum of first N natural numbers using a loop.

#include <iostream>
using namespace std;

```
int main() {  
    long long N, sum = 0;
```

```
    cin >> N;
```

```
    for (long long i = 1; i <= N; i++) {
```

```
        sum += i;
```

```
    }
```

```
    cout << sum;
```

```
    return 0;
```

```
}
```

Explanation

- The loop executes N times
- ↳ Each iteration performs one addition.
- ↳ Only two variables (i and sum) are used.

Optimised approach

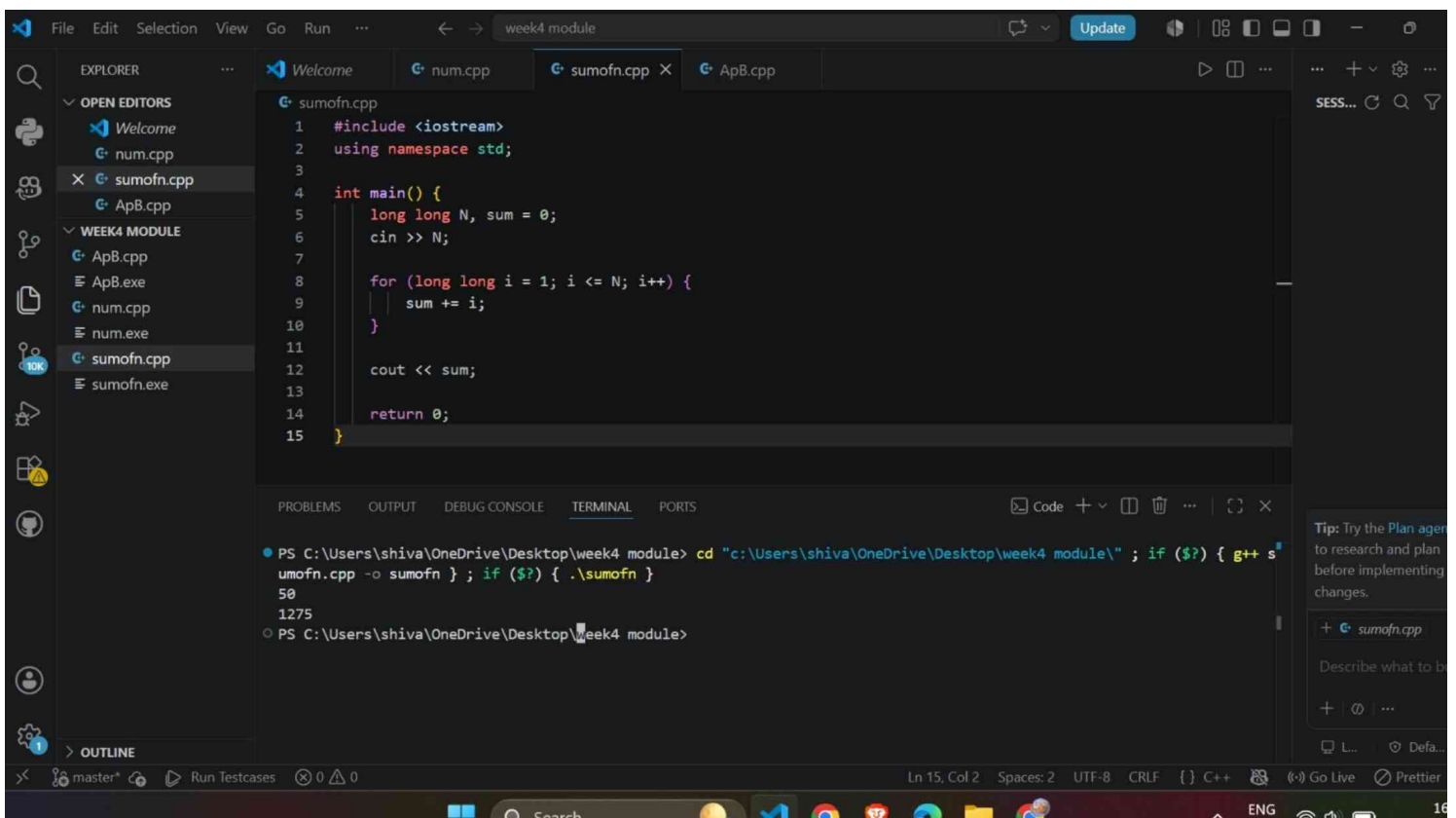
we can use this formula

$$sum = N * (N + 1) / 2;$$

↳ Time complexity: $O(1)$

↳ Space complexity: $O(1)$

Teacher's Signature.....



③ Calculate A^B using a loop.

```
#include <iostream>
using namespace std;

int main() {
    long long A, B;
    cin >> A >> B;
    long long result = -1;
    for (int i = 1; i <= B; i++) {
        result *= A;
    }
    cout << result;
    return 0;
}
```

Time complexity

$O(B)$

Space complexity

$O(1)$

Explanation

- ↳ The loop runs B times
- ↳ Each iteration performs one multiplication.
- ↳ Only a few variables are used, so the extra memory remaining is constant.

Teacher's Signature.....

optimised approach.

use Binary Exponentiation (fast power)

- Time complexity: $O(\log B)$
- Space complexity: $O(1)$

